

Лабораторная работа №7: Основы UML

Цель:

Целью данной работы является знакомство с базовыми приёмами проектирования систем и процессов с использованием универсального языка моделирования (UML)

Задание:

Необходимо разработать комплект UML диаграмм для разрабатываемой на протяжении семестра программы:

1. Диаграмму прецедентов.
2. Диаграмму деятельности.
3. Диаграмму последовательности.
4. Диаграмму классов.

Справочная информация:

Для выполнения лабораторной работы может быть использован свободно распространяемый редактор StarUML, доступный по адресу: <http://staruml.io> или любой другой редактор UML диаграмм.

Диаграмма прецедентов, представляет собой простой способ визуального представления основных возможностей разрабатываемого программного обеспечения или процесса.



Пример диаграммы прецедентов, описывающий функции текстового редактора.

Для описания функций используются действующее лицо (обозначенное человекоподобной фигурой) прецеденты (обозначенные овалами) и ассоциативные связи.

Следует отметить, что действующим лицом может являться не только пользователь, но и программа, программист, специалист службы технической поддержки и т.д. А одной диаграмме, может присутствовать множество действующих лиц.

Помимо ассоциативных связей, существуют:

- направленные ассоциации (линия со стрелкой) – в явной форме указывают характер отношений между прецедентами
- зависимости – указывают на зависимости между прецедентами
- обобщения – указывают на вхождение частного прецедента в более общий



В данном примере описан прецедент расширенной функции “сохранить файл”. Для указания того, что сохранение файла с выбором формата содержит в себе выбор формата и является расширением стандартной функции сохранения файла, используется отношение зависимости.



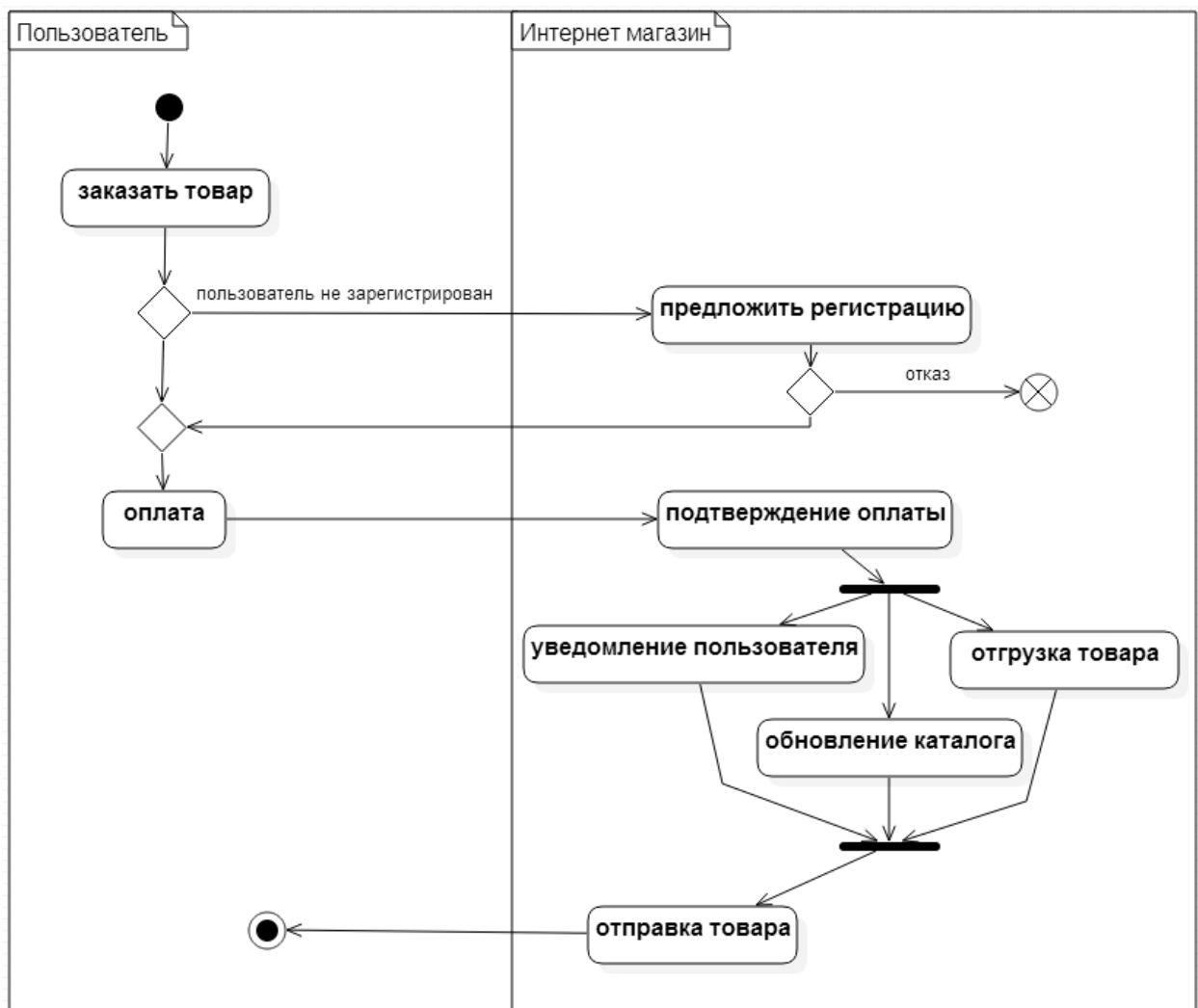
На данной диаграмме показано что ввод текста является частным случаем редактирования файла, а ввод связан с выбором шрифта (без указания типа связи).

Следует помнить, что диаграмма прецедентов должна описывать общие функциональные возможности разрабатываемой программы или системы, а так же фундаментальные зависимости между прецедентами, не акцентируя внимания на деталях реализации процессов и действующих лиц.

Диаграмма деятельности является альтернативой представлению процессов в виде блок схем и используется для описания последовательности действий и выборов.

Состоит из следующих элементов:

- начало процесса – обозначает старт описываемого процесса, может не совпадать с началом работы программы или глобального процесса..
- действие – содержит в себе описание действий на текущем этапе выполнения алгоритма.
- решение – как и на блок схемах, обозначается ромбом, однако не содержит в себе текста. Текст условий ветвления указывается на исходящих из решения управляющих потоках.
- управляющий поток – указывает последовательность выполнения действий.
- разделение – начало блока независимых операций.
- соединение – завершение блока независимых операций.
- завершение процесса – окончание описываемого процесса, может не совпадать с окончанием работы программы или глобального процесса.



Пример описания процесса заказа товара через интернет магазин, при помощи диаграммы деятельности.

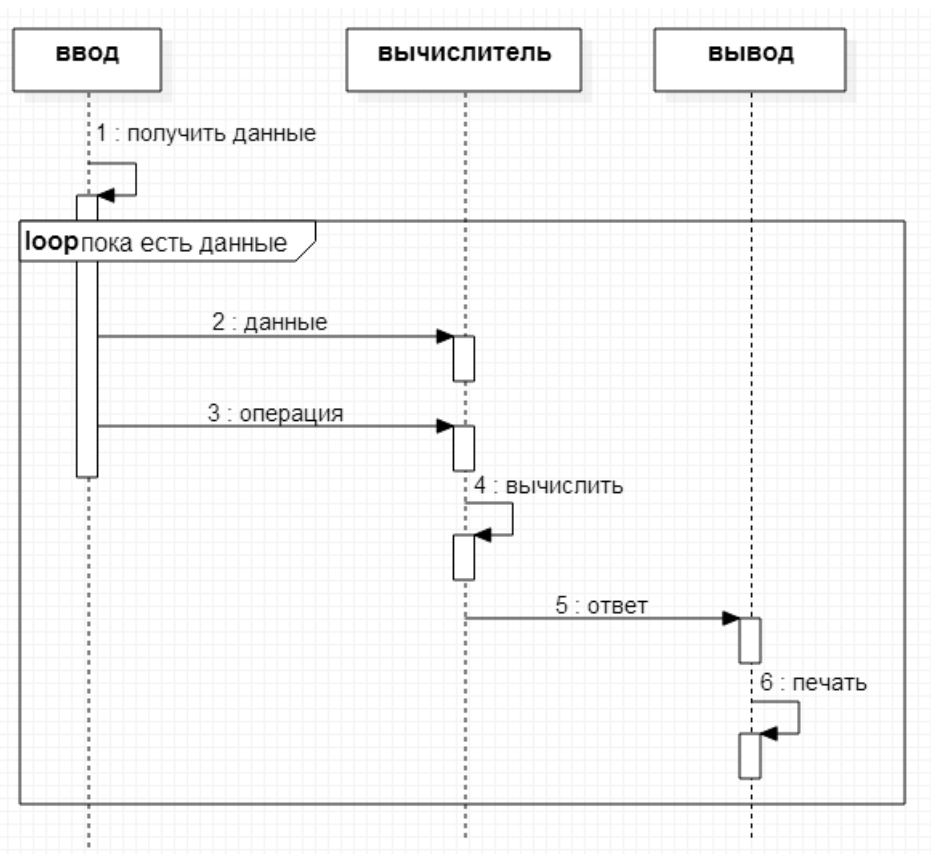
На представленной диаграмме, действия расположены в двух областях, обозначающих действующих лиц участвующих в процессе заказа. Подобное представление не является обязательным при составлении диаграммы деятельности, однако, в дальнейшем, может упростить создание диаграммы последовательности.

Предполагается что операции по уведомлению пользователя, обновлению каталога и отгрузки товара, могут осуществляться одновременно, поскольку независимы друг от друга.

Действие “отправка товара” является обобщённым и может быть представлено в виде отдельной диаграммы, если это необходимо для понимания моделируемых процессов.

Следует помнить, что диаграмма деятельности должна описывать последовательность действий и выборов в процессе выполнения некоего процесса, не акцентируя внимания на классах, полях и методах.

Диаграмма последовательности служит для выявления классов и определения наборов их методов, а так же описания их взаимодействия в процессе выполнения того или иного действия.



На рисунке выше, приведён пример описания процесса работы простого вычислителя арифметических выражений при помощи диаграммы последовательностей.

В самом верху диаграммы изображены задействованные в процессе классы. От них, пунктиром, отмечены их линии жизни.

Прямоугольником на линии жизни объекта обозначается фокус управления. Фокус управления обозначает этап активной работы объекта.

Стрелками указаны сообщения посылаемые объектом. Стрелки, указывающие сами на себя, обозначают методы, меняющие состояние объекта. Стрелки, ведущие к линиям жизни других объектов, обозначают вызов методов этих объектов с некоторыми параметрами.

Рамка с подписью “loop” в левом верхнем углу, называется комбинированным фрагментом и обозначает, что действия в её границах выполняются циклично, до тех пор пока не кончатся данные поступившие на вход.

Существуют следующие виды фреймов взаимодействия:

Alt – условие. Содержимое фрагмента выполняется, только если условие истинно.

Loop – цикл. Содержимое фрагмента будет выполняться, пока не выполнено условие.

Neg – ошибка. Выполняется в случае возникновения ошибки в последовательности.

Opt – условие без альтернативного варианта. Аналогично Alt, но без варианта else.

Par – параллелизм. Фрагменты выполняются (могут выполняться) одновременно.

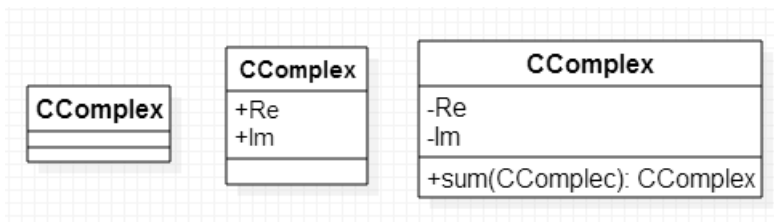
Ref – ссылка. Ссылка на последовательность, описанную в других диаграммах.

Region – критическая секция. Описывает ограничения действующие на время выполнения фрагмента.

Sd – описание. Может охватывать всю диаграмму и содержать некие комментарии.

Следует помнить, что диаграмма последовательности описывает набор классов и способы их взаимодействия, не делая акцент на деталях их реализации.

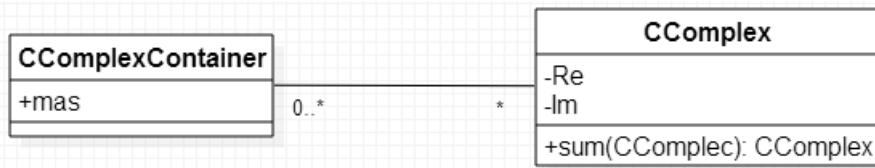
Диаграмма классов служит для детального описания классов и их взаимодействия. Как правило, диаграммы классов могут иметь множество уровней детализации, содержать как некие абстрактные классы и интерфейсы, так и описывать списки полей и методов каждого конкретного объекта описанного в разрабатываемой модели.



Классы, на диаграмме классов, могут быть представлены, как показано на изображении выше.

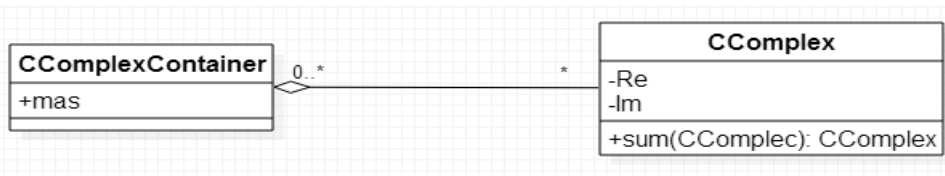
Перед названиями полей и методов указывается модификатор доступа:

- +** - видим для всех (public)
- #** - видим только в классах наследниках (protected)
- - не видим за пределами класса (private)
- ~** - обозначает атрибут с областью видимости типа пакетный (package). Атрибут с этой областью видимости недоступен или не виден для всех классов за пределами пакета, в котором определен класс владелец данного атрибута.

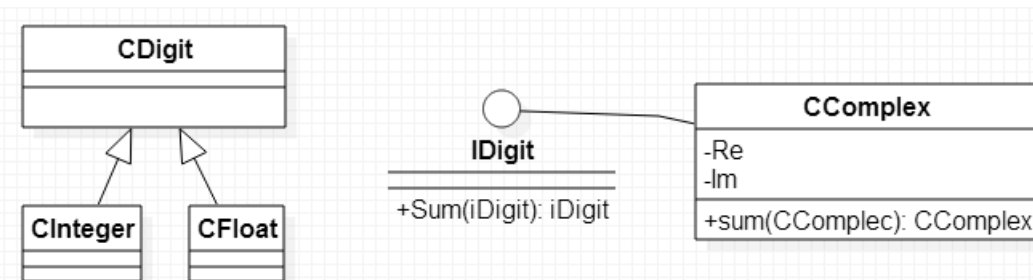


Пример ассоциативной связи между двумя классами.

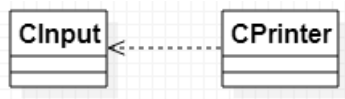
Подобная связь означает, что в состав класса **CComplexContainer** входит множество объектов типа **CComplex**.



Пример связи типа “агрегация”.



Пример описания иерархии наследования от класса и от интерфейса соответственно.



Пример обозначения зависимости между классами. В данном случае, класс CPrinter зависит от класса CInput.

При помощи этого, довольно простого набора обозначений, можно довольно детально описать практически любую систему.

Список литературы:

В данной лабораторной работе, описаны только самые основные моменты составления некоторых видов UML диаграмм. Для того что бы ознакомиться с полным списком возможностей данного языка, рекомендуется самостоятельно ознакомиться с тематической литературой.

1. Нотация и семантика языка UML: <http://www.intuit.ru/studies/courses/32/32/info>